

ARQUITECTURA Y SISTEMAS OPERATIVOS

Trabajo Práctico: Seguridad en sistemas operativos



Alumnos:

Rodrigo Agüero - aguerorodrigo91@gmail.com

Federico Villalba Petrelli - federico.villalba.petrelli@gmail.com

Materia: Arquitectura y Sistemas Operativos

Profesor: Osvaldo Falabella

Tutor: Patricio Costello

Fecha de Entrega: 22 de Junio de 2025

Índice

Sección	Página
1. Introducción	3
2. Marco Teórico	4
3. Caso Práctico	8
3.1. Definición de Roles y Objetivos	8
3.2. Implementación Paso a Paso	8
4. Metodología Utilizada	19
5. Resultados Obtenidos	20
6. Conclusiones	21
7. Bibliografía	22
8. Anexos	23

1. Introducción:

La seguridad en sistemas operativos es un pilar fundamental en el entorno informático actual, donde la integridad y confidencialidad de los datos deben ser garantizadas ante múltiples amenazas. En este trabajo se abordarán los principales mecanismos de protección en entornos Linux, haciendo foco en la gestión de permisos y en la implementación de un modelo de control de acceso basado en roles (RBAC).

El **RBAC** (Role-Based Access Control), en español **Control de Acceso Basado en Roles**, es una técnica para gestionar permisos de manera eficiente, asignando roles a los usuarios en lugar de permisos individuales. Esto simplifica la administración y mejora la seguridad del sistema al agrupar permisos en roles específicos según las funciones que cada usuario cumple dentro del sistema.



Se eligió este tema debido a su relevancia en cualquier sistema informático: tanto desarrolladores, administradores de sistemas, como usuarios finales dependen de un entorno seguro para operar con confianza. La temática es particularmente importante para la formación como técnico en programación, ya que permite comprender las bases de la seguridad desde el sistema operativo y aplicar buenas prácticas en el desarrollo de software.

El objetivo del trabajo es analizar e implementar un entorno de seguridad simulado en Linux, aplicando principios de RBAC mediante el uso de grupos y permisos. A través de esta práctica, se busca replicar una estructura similar a la que proporciona Windows con sus **Listas de Control de Acceso (ACL)**, utilizando los mecanismos de permisos propios de Linux.

Se espera que al finalizar el trabajo, seamos capaces de comprender cómo administrar accesos de forma estructurada y segura en un sistema operativo de código abierto.

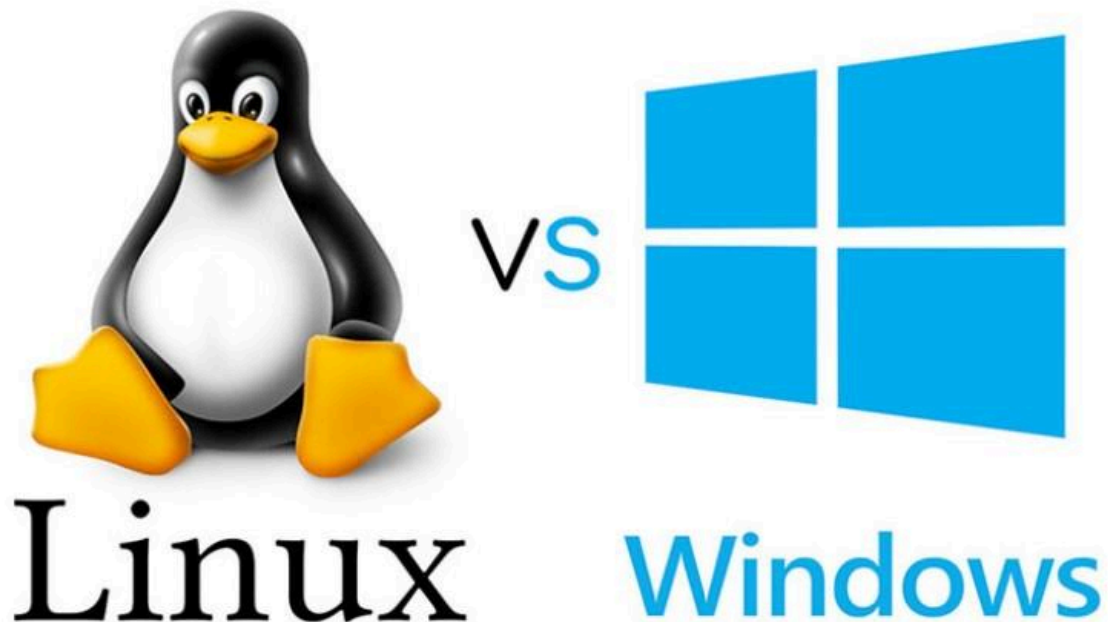
2. Marco Teórico:

La seguridad en los sistemas operativos es el conjunto de mecanismos que permiten proteger tanto los recursos del sistema como la información de los usuarios. Estos mecanismos se aplican a distintos niveles, como la protección de archivos, la gestión de usuarios y permisos, y los procesos de autenticación. En los tiempos actuales, donde los ataques informáticos son cada vez más frecuentes y sofisticados, es fundamental conocer los riesgos a los que están expuestos los sistemas y aplicar estrategias que permitan reducir sus vulnerabilidades.



Todo dispositivo conectado a una red representa una posible puerta de entrada para un atacante. Por eso, es importante estar atentos a las amenazas más comunes, como el malware, la explotación de fallos en el software (exploits), los ataques por fuerza bruta o incluso los intentos de phishing. El impacto de estos riesgos puede ser muy alto: desde la pérdida total o parcial de datos, la interrupción de servicios esenciales, hasta el acceso no autorizado a información confidencial, que es justamente uno de los aspectos centrales de este trabajo práctico integrador.

En cuanto a los sistemas operativos, **Linux** y **Windows** presentan diferencias significativas en términos de seguridad, arquitectura y gestión de permisos.



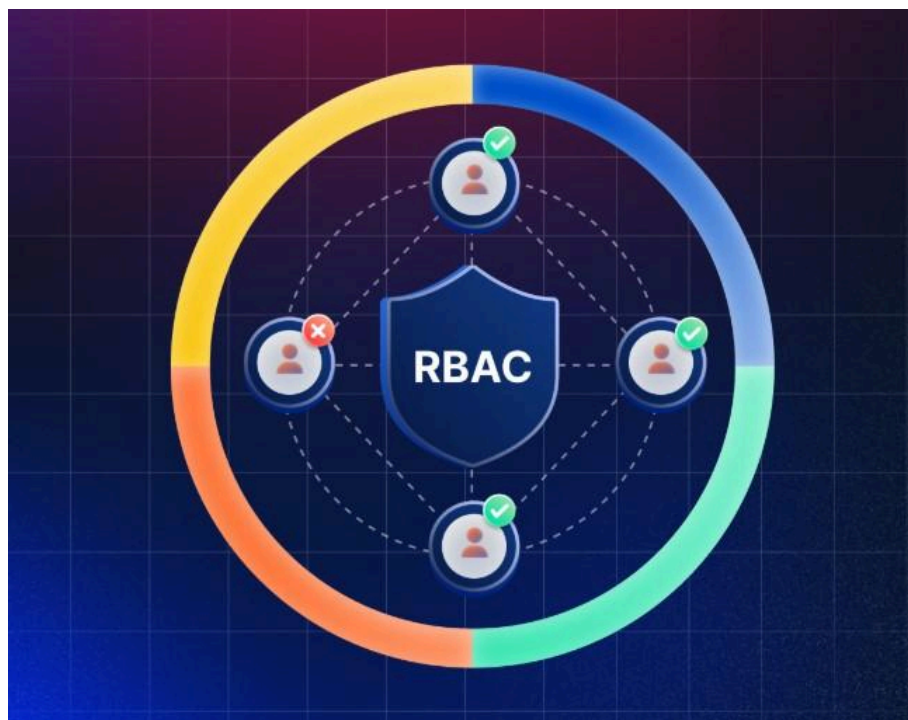
Linux se basa en una filosofía de código abierto, lo que permite que la comunidad detecte y corrija vulnerabilidades de forma más rápida. Además, su modelo de permisos predeterminado (**lectura, escritura y ejecución** para **propietario, grupo y otros**) facilita una gestión sencilla pero eficaz del acceso a archivos y recursos del sistema.

Windows, en cambio, utiliza un enfoque más centralizado y cerrado, con herramientas como Directivas de Grupo (**GPO**) y las **Listas de Control de Acceso (ACL)**, integradas al sistema de archivos **NTFS**. Esta flexibilidad conlleva también una mayor complejidad en la administración de la seguridad.

En cuanto al uso, Windows es más común en entornos de escritorio y en usuarios finales, mientras que Linux predomina en servidores, sistemas de desarrollo y entornos donde la estabilidad, personalización y seguridad son prioritarias.

Estas diferencias hacen que Linux sea percibido como más seguro por defecto, aunque ambos sistemas pueden ser vulnerables si no se configuran correctamente.

Por tal motivo, en este trabajo práctico integrador, buscaremos emular el funcionamiento de las **ACL** de **Windows**, mediante la implementación de **RBAC** en **Linux**, utilizando herramientas nativas como la gestión de usuarios, grupos y permisos de archivos.



Esto permite replicar una estructura de control de acceso organizada y eficaz, adaptada al entorno Linux.

El **Control de Acceso Basado en Roles** permite asignar permisos de manera más eficiente, evitando errores y simplificando la administración. En lugar de definir permisos individuales para cada usuario, RBAC agrupa permisos según funciones específicas como **administrador**, **editor** o **lector**, y asigna esos roles a los usuarios.



Aunque Linux no ofrece una implementación de RBAC tan formal como Windows, es posible simularlo mediante la creación de grupos de usuarios y la asignación precisa de permisos sobre archivos y directorios.

Estos conceptos forman la base sobre la cual se desarrolla el caso práctico de este trabajo, con el objetivo de diseñar un entorno Linux seguro y estructurado, en el que los usuarios accedan únicamente a los recursos necesarios según su rol, replicando así de forma sencilla la lógica de seguridad implementada en sistemas Windows.

3. Caso Práctico:

Simulación de ACL de Windows en Linux usando RBAC

El objetivo principal fue crear un entorno simulado en **Linux** que permita implementar un esquema de Control de Acceso Basado en Roles (**RBAC**), para poder emular el comportamiento de las Listas de Control de Acceso (**ACL**) de **Windows**.

3.1 Definición de Roles y Objetivos

Se definieron tres roles funcionales, cada uno asociado a un grupo del sistema operativo:

- **Administrador** -> grupo: admin
- **Editor** -> grupo: editor
- **Lector** -> grupo: reader

Cada rol fue representado por un **grupo de usuario**, con acceso limitado a su propio conjunto de archivos y carpetas.

3.2 Implementación Paso a Paso

Paso 1: Creamos los Grupos:

Se crearon tres grupos en el sistema para representar cada rol:

```
fvp@MiniPC:~$ sudo groupadd admin
[sudo] password for fvp:
groupadd: group 'admin' already exists
fvp@MiniPC:~$ sudo groupadd editor
fvp@MiniPC:~$ sudo groupadd reader
```

Paso 2: Creamos los usuarios y asignamos a grupos:

Se crearon tres usuarios, uno para cada grupo:

```
fvp@MiniPC:~$ sudo useradd -m -G admin admin_user
fvp@MiniPC:~$ sudo useradd -m -G editor editor_user
fvp@MiniPC:~$ sudo useradd -m -G reader reader_user
```

Con el comando **cat /etc/passwd** podemos ver los usuarios creados anteriormente:

```
admin_user:x:1001:1003:~/home/admin_user:/bin/sh
editor_user:x:1002:1004:~/home/editor_user:/bin/sh
reader_user:x:1003:1005:~/home/reader_user:/bin/sh
```

Paso 3: Creamos la estructura de directorios:

Se creó la carpeta principal del proyecto y subdirectorios por rol:

```
fvp@MiniPC:~$ sudo mkdir -p /proyecto/admin
[sudo] password for fvp:
fvp@MiniPC:~$ sudo mkdir -p /proyecto/editor
fvp@MiniPC:~$ sudo mkdir -p /proyecto/reader
```

Se generaron los archivos de prueba en cada directorio:

```
fvp@MiniPC:~$ sudo touch /proyecto/admin/datos_admin.txt
fvp@MiniPC:~$ sudo touch /proyecto/editor/datos_editor.txt
fvp@MiniPC:~$ sudo touch /proyecto/reader/datos_reader.txt
```

Paso 3.1: Instalamos Paquete Tree:

Se instaló paquete tree para poder ver árbol de directorios:

```
fvp@MiniPC:~$ tree /proyecto
Command 'tree' not found, but can be installed with:
sudo snap install tree # version 2.1.3+pkg-5852, or
sudo apt install tree # version 2.1.1-2
See 'snap info tree' for additional versions.
fvp@MiniPC:~$ sudo apt install tree
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  tree
0 upgraded, 1 newly installed, 0 to remove and 42 not upgraded.
Need to get 47.1 kB of archives.
After this operation, 111 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 tree amd64 2.1.1-2ubuntu3 [47.1 kB]
Fetched 47.1 kB in 1s (57.8 kB/s)
Selecting previously unselected package tree.
(Reading database ... 40769 files and directories currently installed.)
Preparing to unpack .../tree_2.1.1-2ubuntu3_amd64.deb ...
Unpacking tree (2.1.1-2ubuntu3) ...
Setting up tree (2.1.1-2ubuntu3) ...
Processing triggers for man-db (2.12.0-4build2) ...
```

Se ejecutó comando para poder ver árbol de directorio que creamos anteriormente:

```
fvp@MiniPC:~$ tree /proyecto
/proyecto
├── admin
│   └── datos_admin.txt
├── editor
│   └── datos_editor.txt
└── reader
    └── datos_reader.txt

4 directories, 3 files
```

Paso 4: Asignamos los permisos y propietarios:

Una vez creados los directorios y los archivos, se procede a asignar **propietarios** y **permisos** adecuados para asegurar que cada usuario solo acceda a su espacio asignado.

- Asignamos propietario y grupo a cada carpeta:

```
sudo chown -R admin_user:admin /proyecto/admin  
sudo chown -R editor_user:editor /proyecto/editor  
sudo chown -R reader_user:reader /proyecto/reader
```

Esto hace que cada carpeta y su archivo pertenezcan al usuario y grupo correcto según el rol

- Asignamos permisos a las carpetas:

```
sudo chmod 770 /proyecto/admin /proyecto/editor /proyecto/reader
```

A cada directorio se le otorgaron permisos **770**, lo que permite acceso completo al propietario y al grupo, y ninguno a otros usuarios.

Con esto nos aseguramos que **propietario** y **grupo** pueden leer, escribir y ejecutar.

Los demás usuarios no tienen ningún acceso.

- Asignamos permisos a los archivos:

```
sudo chmod 660 /proyecto/admin/datos_admin.txt
sudo chmod 660 /proyecto/editor/datos_editor.txt
sudo chmod 660 /proyecto/reader/datos_reader.txt
```

A cada archivo se le otorgaron permisos **660**, lo que permite al propietario y al grupo poder leer y escribir el archivo pero a ningún otro usuario.

En palabras más simples, concedemos permisos de Lectura y Escritura al propietario y grupo.

Y ningún permiso para otros usuarios.

- Verificamos correcta asignación de permisos:

```
fvp@MiniPC:~$ sudo ls -l /proyecto/*/
/proyecto/admin/:
total 0
-rw-rw---- 1 admin_user admin 0 Jun 18 21:24 datos_admin.txt

/proyecto/editor/:
total 0
-rw-rw---- 1 editor_user editor 0 Jun 18 21:24 datos_editor.txt

/proyecto/reader/:
total 0
-rw-rw---- 1 reader_user reader 0 Jun 18 21:24 datos_reader.txt
```

Con esta salida podemos observar que todo está configurado como queríamos.

- ❖ Cada archivo está ubicado en su carpeta correspondiente.
- ❖ Cada archivo pertenece al **grupo** correcto (admin, editor, reader).
- ❖ Cada archivo tiene como **propietario** al usuario del **rol** correcto.
- ❖ Los permisos **660** (**-rw- rw- - -**) están bien aplicados como lectura (**r**) y escritura (**w**) solo para el **usuario propietario** y su **grupo**.

Con todos los pasos anteriores, conseguimos tener la estructura RBAC en Linux simulando ACL de Windows.

Ahora, como plus, vamos generar un script que resume todos los pasos anteriores en 1 sólo archivo para poder ejecutarlo y así automatizar todo el proceso.

Para hacerlo más entendible y didáctico lo pondremos como Paso 5 pero en realidad sería un Paso alternativo a los demás.

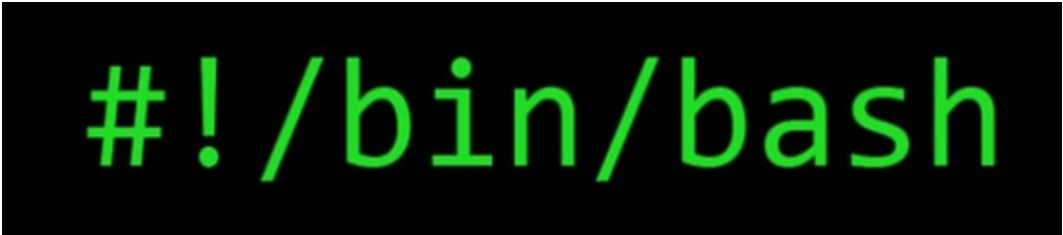
Paso 5: Generamos un Script para Automatizar el proceso:

Para facilitar la implementación y garantizar la replicabilidad del entorno RBAC simulado, se desarrolló un script de automatización llamado **rbac_setup.sh**. Este script ejecuta todos los pasos necesarios para crear los grupos, usuarios, estructura de directorios y asignar permisos de manera ordenada y sin errores manuales.

El objetivo es permitir que cualquier usuario pueda generar el mismo entorno de control de acceso con un solo comando.

Automatizar la creación de grupos, usuarios y asignación de permisos no solo agiliza la implementación, sino que también representa una buena práctica en entornos reales, donde los despliegues deben ser reproducibles y controlados.

La primera línea del script es la siguiente:



```
#!/bin/bash
```

Repasando unidades anteriores, podemos estar en condiciones de identificar dicha línea, a la cual llamamos **shebang**. Lo que hace es indicarle al sistema con qué intérprete debe ejecutar el script. En este caso, le está diciendo que use Bash, que es el shell por defecto en la mayoría de las distribuciones Linux.

Este script puede ejecutarse en cualquier sistema Linux compatible (por ejemplo, Ubuntu), y debe correrse con privilegios de administrador para funcionar correctamente.

Contenido del Script:

```
1  #!/bin/bash
2
3  # Paso 1:
4  # Crear grupos
5  sudo groupadd admin
6  sudo groupadd editor
7  sudo groupadd reader
8
9  # Paso 2:
10 # Crear usuarios y asignarlos a grupos
11 sudo useradd -m -g admin admin_user
12 sudo useradd -m -g editor editor_user
13 sudo useradd -m -g reader reader_user
14
15 # Paso 3:
16 # Crear estructura de directorios
17 sudo mkdir -p /proyecto/admin /proyecto/editor /proyecto/reader
18
19 # Crear archivos de prueba
20 sudo touch /proyecto/admin/datos_admin.txt
21 sudo touch /proyecto/editor/datos_editor.txt
22 sudo touch /proyecto/reader/datos_reader.txt
23
24 # Paso 4:
25 # Asignar propietarios y grupos correctos
26 sudo chown -R admin_user:admin /proyecto/admin
27 sudo chown -R editor_user:editor /proyecto/editor
28 sudo chown -R reader_user:reader /proyecto/reader
29
30 # Asignar permisos a directorios
31 sudo chmod 770 /proyecto/admin /proyecto/editor /proyecto/reader
32
33 # Asignar permisos a archivos
34 sudo chmod 660 /proyecto/admin/datos_admin.txt
35 sudo chmod 660 /proyecto/editor/datos_editor.txt
36 sudo chmod 660 /proyecto/reader/datos_reader.txt
37
38 echo "Configuración de roles y permisos completada."
```

Para ejecutar el script, simplemente hacemos lo siguiente:

1. Nos movemos a la carpeta que contiene el script.

En este caso el archivo se llama **rbac_setup.sh** y lo tenemos ubicado en la carpeta fvp dentro del home.

2. Nos aseguramos que el archivo tenga los permisos de ejecución.

Utilizamos el comando **chmod +x**

3. Ejecutamos el archivo como **superusuario** porque este script crea usuarios, cambia permisos y demás como vimos en los pasos previos.

```
fvp@MiniPC:~$ cd /home/fvp
fvp@MiniPC:~$ chmod +x rbac_setup.sh
fvp@MiniPC:~$ sudo ./rbac_setup.sh
[sudo] password for fvp:
groupadd: group 'admin' already exists
groupadd: group 'editor' already exists
groupadd: group 'reader' already exists
useradd: user 'admin_user' already exists
useradd: user 'editor_user' already exists
useradd: user 'reader_user' already exists
Configuración de roles y permisos completada.
```

Este enfoque permite automatizar todo el proceso y es una buena práctica para ambientes reales, ya que facilita la replicabilidad y evita errores manuales.

Paso 6: Verificamos accesos:

Una vez ejecutado el script y finalizada la configuración, se procedió a verificar que el entorno funcione como se esperaba. Para eso, se inició sesión con cada usuario creado y se realizaron distintas pruebas de acceso.

Se comprobó lo siguiente:

- `admin_user` pudo acceder a `/proyecto/admin` y modificar el archivo `datos_admin.txt`
- `editor_user` tuvo acceso solamente a `/proyecto/editor` y al archivo `datos editor.txt`
- `reader_user` pudo acceder exclusivamente a su carpeta `/proyecto/reader`

Además, se realizaron intentos de lectura y escritura como por ejemplo, `reader_user` intentando leer `datos_admin.txt` y el sistema correctamente denegó el acceso, gracias a los permisos previamente asignados.

Estas pruebas demostraron que la implementación fue exitosa: los usuarios sólo acceden a los recursos definidos por su rol, replicando el comportamiento de un sistema RBAC. Esto valida que tanto la estructura de directorios, los grupos, los permisos y el script de automatización están funcionando correctamente.

Capturas de pantalla de pruebas realizadas:

Aquí asignamos una clave a nuestro usuario admin.

Luego ingresamos con el usuario admin.

Posteriormente ingresamos a la carpeta del usuario correctamente

Y finalmente modificamos el archivo dentro de esa carpeta.

```
fvp@MiniPC:~$ sudo passwd admin_user
New password:
Retype new password:
passwd: password updated successfully
fvp@MiniPC:~$
fvp@MiniPC:~$ su - admin_user
Password:
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 5.15.167.4-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Jun 19 01:03:49 UTC 2025

System load:  0.0               Processes:            36
Usage of /:   0.2% of 1006.85GB  Users logged in:     2
Memory usage: 3%               IPv4 address for eth0: 172.17.3.51
Swap usage:   0%

This message is shown once a day. To disable it please create the
/home/admin_user/.hushlogin file.
$ cd /proyecto/admin
$ cat datos_admin.txt
$ echo "Prueba" >> datos_admin.txt
$ cat datos_admin.txt
Prueba
$
```

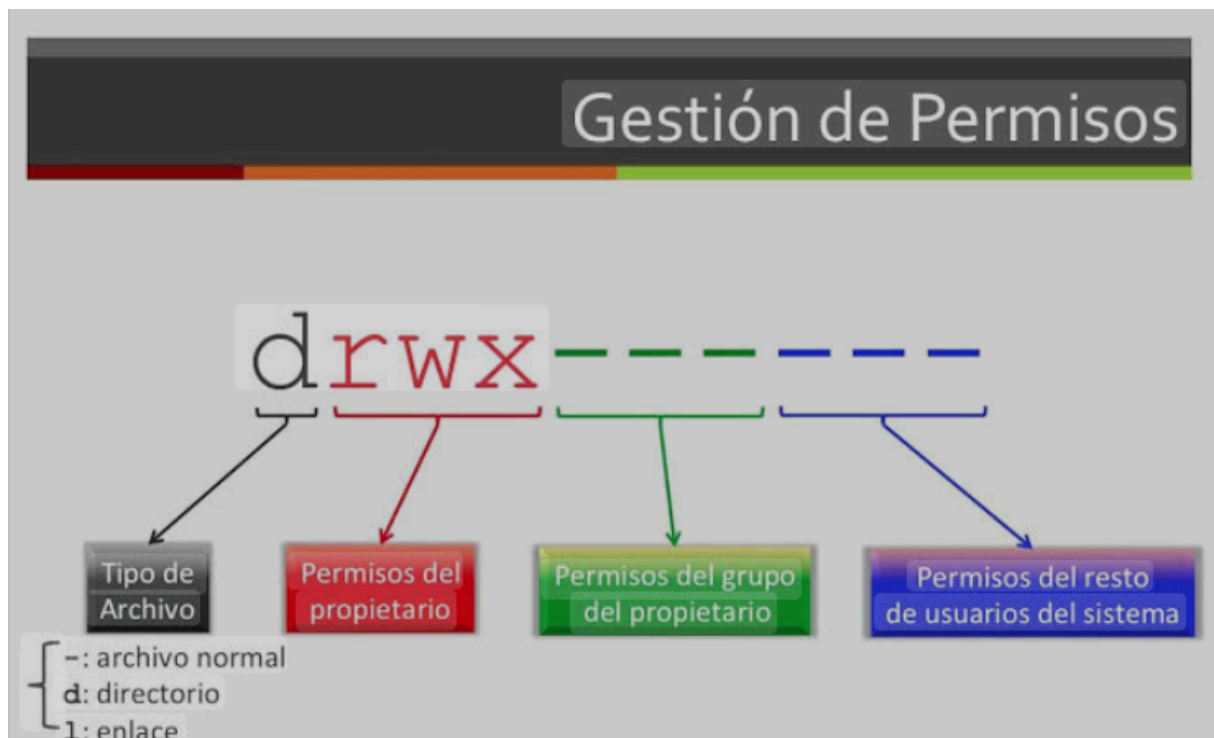
Lo anterior demuestra que el usuario pudo ingresar correctamente a su directorio y archivo para modificarlo.

Luego mostramos con el comando cat que el archivo se encuentra modificado.

Aquí intentamos ingresar a la carpeta del usuario editor pero como se esperaba se deniega el acceso al mismo ya que no estamos con el usuario editor. Lo que demuestra la correcta implementación de RBAC en Linux.

```
$ ls -l
total 12
drwxrwx--- 2 admin_user  admin  4096 Jun 18 21:24 admin
drwxrwx--- 2 editor_user editor  4096 Jun 18 21:24 editor
drwxrwx--- 2 reader_user reader  4096 Jun 18 21:24 reader
$ cd /proyecto/editor
-sh: 14: cd: can't cd to /proyecto/editor
```

Así podríamos seguir probando con los diferentes usuarios, pero lo dejamos para que ustedes puedan probar y experimentar por su cuenta verificando que la implementación fue exitosa.



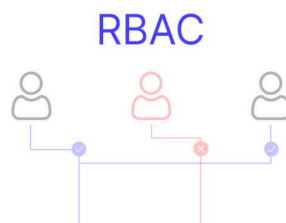
4. Metodología Utilizada:

Para desarrollar este trabajo práctico se siguió una metodología basada en la combinación de investigación teórica, experimentación en entorno controlado y automatización de tareas.

En primer lugar, se realizó una revisión de contenidos relacionados con la seguridad en sistemas operativos, específicamente el modelo de control de acceso basado en roles (RBAC), los permisos en Linux y las diferencias respecto a las ACL en Windows. Esta investigación se apoyó en apuntes de la materia, documentación oficial y fuentes confiables de internet.

Luego, se diseñó un entorno simulado en una máquina virtual con Linux (Ubuntu), en el cual se implementó paso a paso la estructura de grupos, usuarios, carpetas y archivos. Durante esta etapa se realizaron pruebas intermedias para comprobar el comportamiento esperado del sistema y corregir posibles errores de configuración.

Para agilizar la implementación y evitar errores manuales, se desarrolló un script en Bash con un archivo llamado **rbac_setup.sh** que automatiza la creación de los roles, usuarios, directorios y asignación de permisos. Esto permitió aplicar de forma práctica conceptos vistos durante la cursada y fomentar el uso de herramientas que faciliten el trabajo del administrador de sistemas.



Finalmente, se verificó el funcionamiento del entorno accediendo con cada usuario creado, intentando visualizar o modificar archivos ajenos, y comprobando que el sistema respeta las restricciones establecidas por los permisos que les dimos. Esta validación fue clave para confirmar que el esquema RBAC simulado funcionaba correctamente.

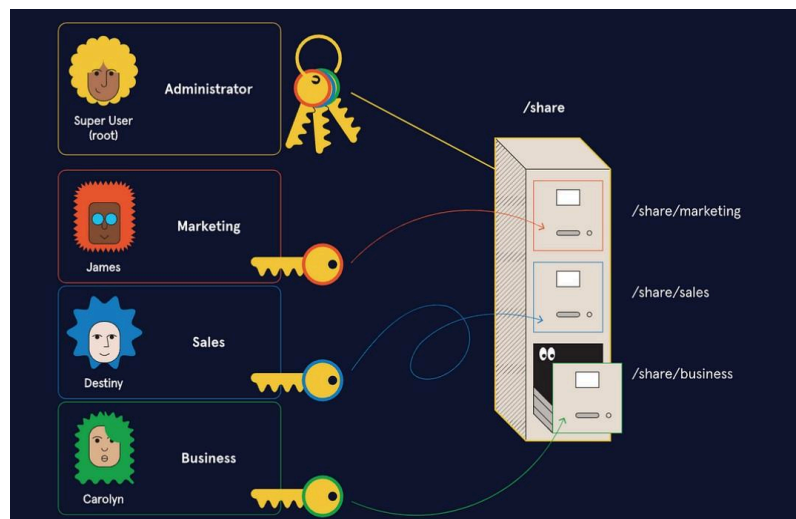
5. Resultados Obtenidos:

Como resultado del trabajo realizado, se logró implementar con éxito un entorno seguro en **Linux** basado en el modelo de Control de Acceso Basado en Roles (**RBAC**). A través de la creación de grupos, usuarios, directorios y archivos con permisos específicos, se replicó el comportamiento de las Listas de Control de Acceso (**ACL**) presentes en **Windows**.

La configuración manual inicial nos permitió entender el funcionamiento detallado de los permisos en Linux, mientras que el desarrollo del script automatizado con el archivo **rbac_setup.sh** facilitó la implementación completa del entorno de forma rápida y sin errores.

Durante la etapa de verificación, se comprobó que:

- Cada usuario puede acceder **únicamente** a los archivos y carpetas asignadas a su rol.
- Se bloquearon correctamente los intentos de acceso no autorizados a recursos que no le pertenecían o no tenían permisos.
- Los permisos aplicados fueron respetados en todos los casos



El entorno se comportó de acuerdo con lo esperado, validando la eficacia del esquema de seguridad implementado. Esto permitió reforzar los conocimientos adquiridos sobre gestión de accesos en sistemas Linux y entender cómo aplicar buenas prácticas de seguridad en entornos reales.

6. Conclusiones:

A lo largo de este trabajo práctico pudimos aplicar de forma concreta los conceptos teóricos relacionados con la seguridad en sistemas operativos, especialmente en lo referido a la gestión de permisos y el control de acceso basado en roles.

La implementación de un entorno **RBAC** en **Linux**, mediante la creación de usuarios, grupos y asignación de permisos, nos permitió simular de manera sencilla y efectiva cómo funcionan las Listas de Control de Acceso (**ACL**) en sistemas **Windows**. Este enfoque resultó útil para organizar el acceso a los recursos del sistema de forma segura y estructurada

El desarrollo del script automatizado también representó un aprendizaje importante, ya que mostró cómo se pueden reducir errores manuales y agilizar tareas repetitivas mediante el uso de herramientas de línea de comandos aprendidos en unidades anteriores. Además, la verificación de los accesos reforzó nuestra comprensión sobre cómo se aplican y respetan los permisos en un sistema Linux correctamente configurado.



En conclusión, este trabajo nos permitió no solo afianzar conocimientos técnicos, sino también valorar la importancia de aplicar buenas prácticas de seguridad desde el sistema operativo, algo fundamental tanto para entornos personales como profesionales.

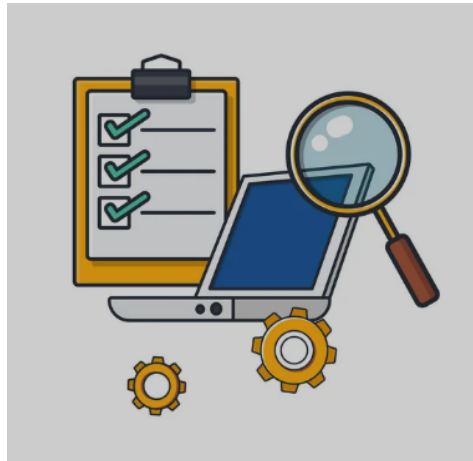
7. Bibliografía:



- Apuntes de la materia Arquitectura y Sistemas Operativos
- PDF y Videos sobre Seguridad en los Sistemas Operativos
- The Linux Command Line (William Shotts)
- <https://wiki.archlinux.org/title/Security>
- <https://linux.die.net/man/>

8. Anexos:

A continuación, se incluyen algunos elementos adicionales que complementan el desarrollo del trabajo práctico:



- Capturas de Pantallas
- Imágenes
- Video Explicativo.
- Código Fuente Completo, se encuentra adjunto en el repositorio:

<https://github.com/RodrigoAguero91/Proyecto-Integrador-AYSO/>

- Script Bash completo en archivo **rbac_setup.sh**
- Comandos utilizados durante la implementación:
 - groupadd, useradd, mkdir, touch, chown, chmod
 - sudo apt install
 - tree, cat, ls -l
 - sudo ./rbac_setup.sh